

UNIVERSIDAD DE SAN MARTÍN DE PORRES FILIAL SUR

FACULTAD DE INGENIERÍA Y ARQUITECTURA



Curso: Sistemas de Información

Docente: Ing. Jeymi Melanie Valdivia

Informe Entregable

Sistema de Gestión Administrativa para Consultas del
Programa Profesional de Ing. Computación y Sistemas FIA

Elaborado por:

Mauricio David Mamani Pacori

Bedoya Chávez Idan Naed

Noe Oliver Vargas Ochoa

Aldair Gabriel Marín Silveira

Nota: _____

Obs:

Índice

1. Portada	1
2. Índice	2
3. Introducción y Objetivos	3
3.1 Introducción	3
3.2 Objetivos	3
3.2.1 Objetivo general	3
3.2.2 Objetivos específicos	3
4. Etapa 1: Análisis de Requerimientos	4
4.1 Requisitos funcionales	4
4.2 Requisitos no funcionales	5
5. Etapa 2: Diseño del sistema	6
5.1 Diagrama de flujo	6
5.2 Modelo Entidad-Relación	7
5.3 Diagrama de casos de uso	7
5.4 Mockups de pantallas	8
6. Etapa 3: Desarrollo	9
6.1 Lenguaje y herramientas usadas	9
6.2 Arquitectura del sistema	10
6.3 Capturas del sistema en ejecución	10
7. Etapa 4: Pruebas realizadas	11
7.1 Escenarios de prueba	11
7.2 Resultados	11
8. Etapa 5: Documentación	12
8.1 Manual técnico (instalación)	12
8.2 Manual de usuario	12
9. Etapa 6: Propuesta de mejoras	13
10. Conclusiones y aprendizajes	14
11. Anexos	15
11.1 Código fuente	15
11.2 Estructura de base de datos	15
11.3 Bibliografía	15

Introducción y Objetivos

1. Introducción

En el entorno universitario, los estudiantes frecuentemente enfrentan situaciones que requieren atención institucional: notas incorrectas, problemas de acceso a sistemas, consultas financieras, entre otras. Actualmente, estas consultas se gestionan por múltiples canales informales como correo electrónico, WhatsApp y atención presencial, lo que genera desorganización, pérdida de información y tiempos de respuesta inconsistentes.

El presente proyecto propone el desarrollo de un **Sistema de Gestión Administrativa para Consultas**, una plataforma web que centraliza el reporte, seguimiento y resolución de solicitudes estudiantiles. El sistema permite que los alumnos registren sus consultas de forma estructurada, mientras que el personal administrativo puede gestionarlas, priorizarlas y responderlas desde un panel dedicado.

Una característica destacada es la **asignación automática de prioridad** mediante análisis de palabras clave, clasificando cada consulta como alta, media o baja según el contenido del reporte. Adicionalmente, el sistema incorpora una **derivación automática al área responsable** según la categoría seleccionada, eliminando la clasificación manual por parte de la secretaria.

El sistema cuenta con un **Catálogo de Servicios Académicos** que permite a los alumnos consultar información detallada de trámites frecuentes (requisitos, procedimiento, responsable y tiempo estimado), reduciendo la necesidad de crear consultas. Además, integra una **Base de Conocimientos (KMS)** que sugiere respuestas automáticas mientras el alumno escribe su consulta, aprendiendo de las respuestas promovidas por el staff desde consultas resueltas.

El panel administrativo incluye **métricas temporales** como consultas por mes y tiempo promedio de resolución, así como **reportes visuales** con gráficas de barras por estado, categoría, prioridad y distribución mensual.

El sistema fue desarrollado con una arquitectura moderna de tres capas: frontend en React con Vite y Tailwind CSS, backend en FastAPI con Python, y base de datos en Supabase (PostgreSQL), desplegado en la nube mediante Vercel y Render.

2. Objetivos

2.1 Objetivo general

Desarrollar un sistema web de registro, seguimiento y resolución de consultas administrativas que centralice la comunicación entre alumnos y el personal de la facultad, permitiendo una atención priorizada, organizada y con base de conocimientos integrada.

2.2 Objetivos específicos

- Permitir a los alumnos registrar consultas indicando categoría, título y descripción del problema, con asignación automática de fecha, hora y código único.
- Automatizar la clasificación de prioridad (alta, media, baja) mediante análisis de palabras clave en el título y descripción de cada consulta.
- Implementar un sistema de autenticación con roles diferenciados (alumno, secretaria, admin), asegurando que cada usuario acceda únicamente a las funciones que le corresponden.
- Proveer al personal administrativo un panel donde pueda visualizar todas las consultas ordenadas por prioridad, cambiar su estado y enviar respuestas al alumno.
- Derivar automáticamente cada consulta al área responsable según su categoría, eliminando la clasificación manual por parte de la secretaria.
- Implementar un Catálogo de Servicios Académicos con información detallada de trámites frecuentes para reducir consultas innecesarias.
- Registrar un historial completo de cambios de estado por cada consulta, con fecha, hora y comentario del responsable.
- Implementar una Base de Conocimientos (KMS) que sugiera respuestas automáticas mientras el alumno escribe, y que aprenda de las consultas resueltas por el staff.
- Generar reportes estadísticos con gráficas de barras y métricas temporales (consultas por mes y tiempo promedio de resolución), filtrados por fecha, estado y categoría.

3. Etapa 1: Análisis de Requerimientos

3.1 Requisitos funcionales

- **RF01:** El sistema debe permitir el registro de usuarios con roles: alumno, secretaria y admin.
- **RF02:** El sistema debe permitir el inicio de sesión mediante credenciales (email y contraseña).
- **RF03:** El sistema debe diferenciar el acceso según el rol del usuario autenticado.
- **RF04:** El alumno debe poder registrar consultas indicando categoría, título y descripción.
- **RF05:** El sistema debe asignar automáticamente fecha, hora y código único (CON-XXXXXXXX) a cada consulta.
- **RF06:** El sistema debe inferir automáticamente la prioridad (alta, media, baja) según palabras clave del contenido.
- **RF07:** El sistema debe clasificar consultas por categoría: académico, financiero, infraestructura, sistemas, matrícula, trámites y otro.
- **RF08:** El alumno debe poder visualizar el estado actual y la respuesta de sus consultas.
- **RF09:** El personal administrativo debe poder actualizar el estado de las consultas con comentarios.
- **RF10:** El sistema debe guardar el historial completo de cambios de estado por consulta.
- **RF11:** El personal debe poder responder consultas, cambiando automáticamente el estado a resuelto.
- **RF12:** El sistema debe mostrar sugerencias del KMS mientras el alumno escribe su consulta.
- **RF13:** El staff debe poder promover respuestas de consultas resueltas como artículos del KMS.
- **RF14:** El sistema debe generar reportes estadísticos por estado, categoría y prioridad.

- **RF15:** El sistema debe permitir filtrar reportes por rango de fechas, estado y categoría.
- **RF16:** El código de alumno (8 dígitos) debe funcionar automáticamente como contraseña inicial.
- **RF17:** El sistema debe disponer de un Catálogo de Servicios Académicos con información detallada de trámites frecuentes.
- **RF18:** El sistema debe derivar automáticamente cada consulta al área responsable según su categoría.
- **RF19:** El sistema debe mostrar el tiempo promedio de resolución y la distribución de consultas por mes en el dashboard administrativo.
- **RF20:** El alumno debe poder consultar el Catálogo de Servicios y la Base de Conocimientos antes de crear una consulta.

3.2 Requisitos no funcionales

- **RNF01:** El sistema debe ofrecer tiempos de respuesta rápidos ante las solicitudes del usuario.
- **RNF02:** El sistema debe garantizar autenticación segura mediante JWT y control de acceso por roles (RBAC).
- **RNF03:** La información sensible debe estar protegida; las credenciales no deben exponerse en el código fuente.
- **RNF04:** La interfaz debe ser intuitiva y fácil de usar para el usuario final sin capacitación previa.
- **RNF05:** El sistema debe ser responsive y compatible con múltiples dispositivos y tamaños de pantalla.
- **RNF06:** El sistema debe estar disponible en la nube, accesible desde cualquier navegador moderno.
- **RNF07:** La arquitectura debe estar separada en capas (frontend, backend, base de datos) para facilitar el mantenimiento.

4. Etapa 2: Diseño del sistema

4.1 Diagrama de flujo

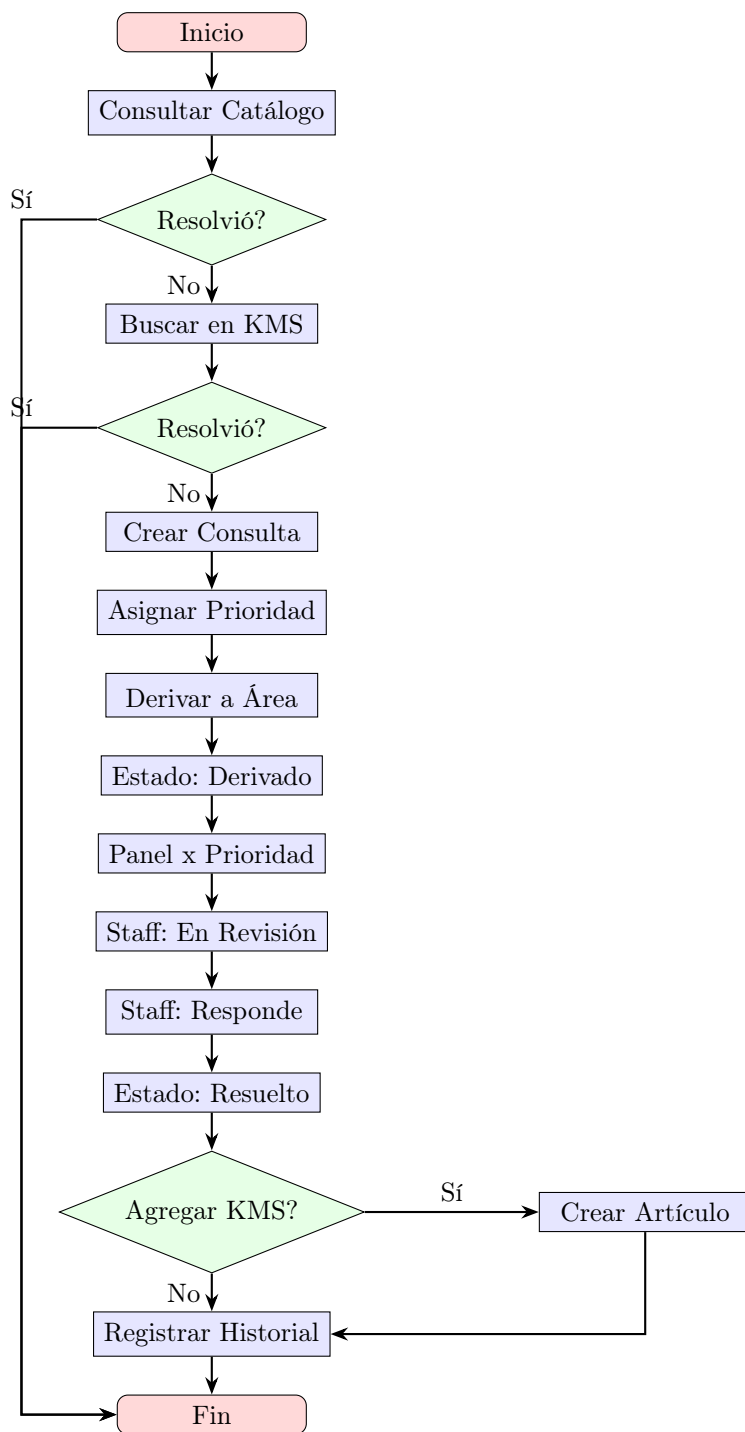


Figura 1: Diagrama de flujo del sistema

4.2 Modelo Entidad-Relación

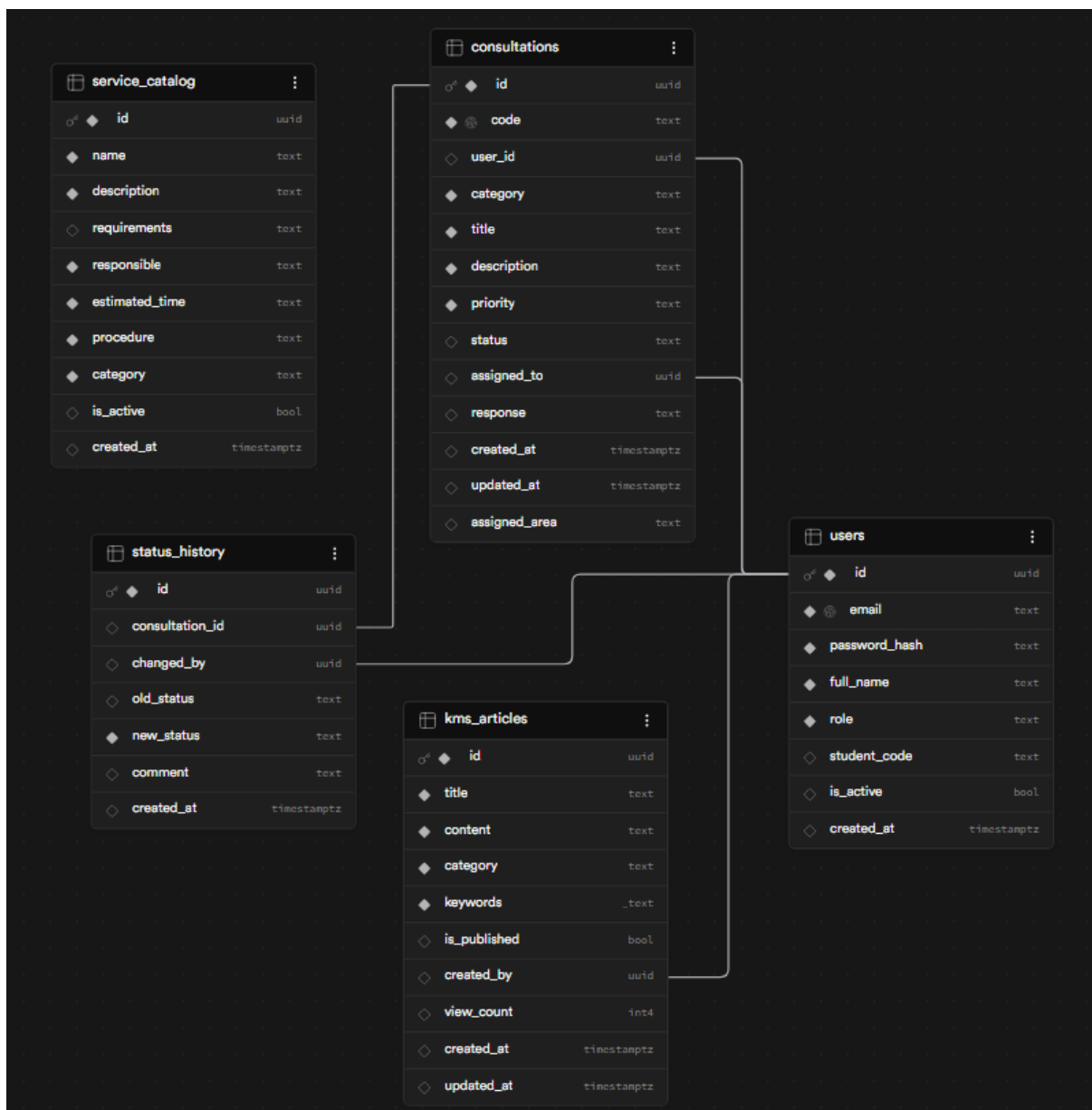


Figura 2: Modelo Entidad-Relación de la base de datos

El modelo contempla 5 entidades principales:

- **users**: Almacena los datos de todos los usuarios del sistema con su rol asignado.
- **service_catalog**: Catálogo de servicios académicos con nombre, descripción, requisitos, procedimiento, responsable y tiempo estimado.
- **consultations**: Registra cada consulta con su código único, prioridad inferida, área derivada y estado actual.

■ **status_history:** Auditoría completa de cada cambio de estado realizado sobre una consulta.

■ **kms_articles:** Base de conocimientos con artículos indexados por palabras clave y categoría.

4.3 Diagrama de casos de uso

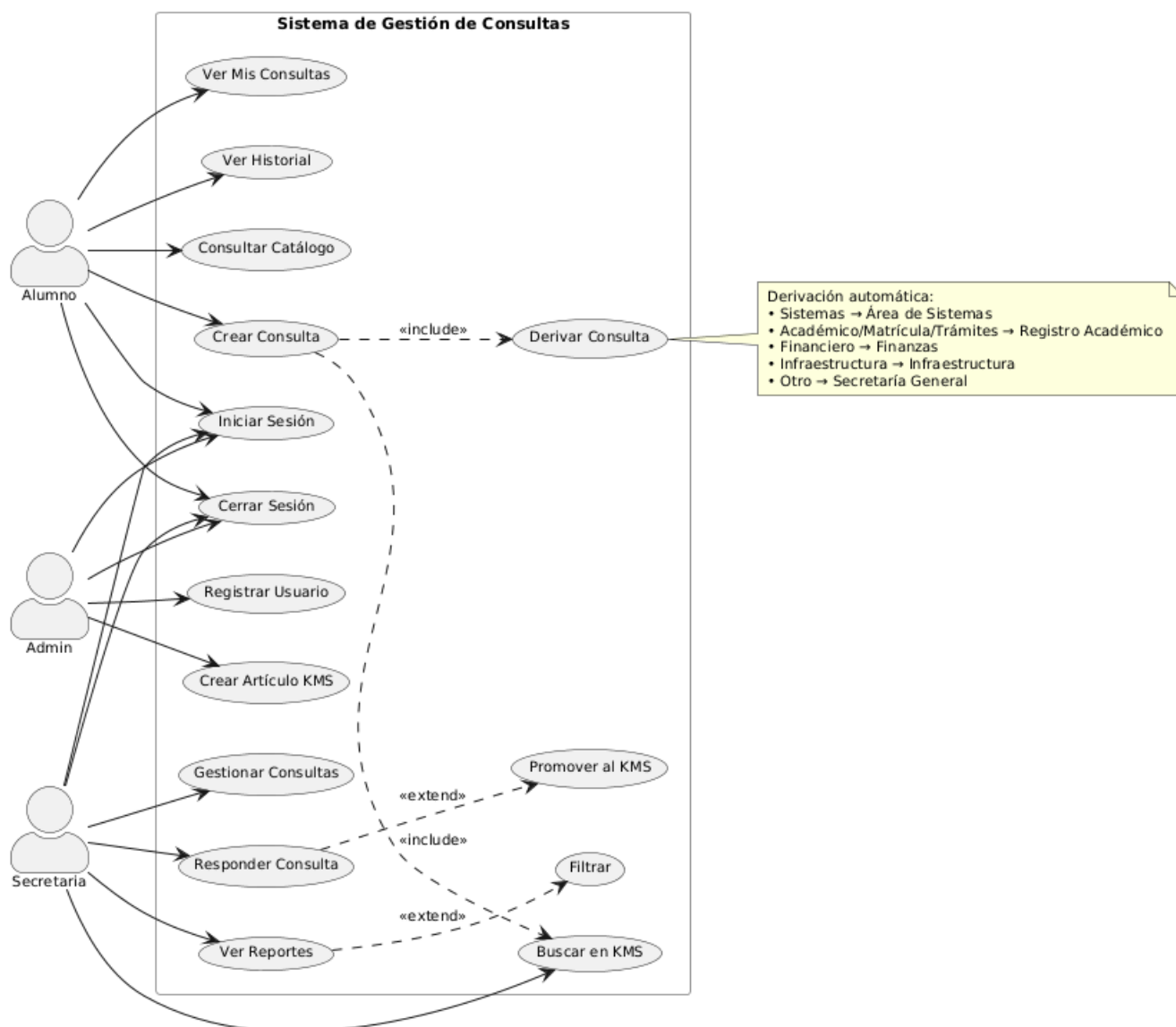


Figura 3: Diagrama de casos de uso del sistema

El diagrama contempla 3 actores y 7 paquetes funcionales:

■ **Autenticación:** Inicio y cierre de sesión, común para todos los roles.

- **Catálogo de Servicios:** Consulta de servicios académicos con información detallada de cada trámite.
- **Consultas:** Crear consulta, ver el historial y las respuestas del staff.
- **Gestión de Consultas:** Funciones del staff: ver, filtrar, cambiar estado y responder consultas.
- **Base de Conocimientos (KMS):** Buscar, crear y promover artículos desde consultas resueltas.
- **Reportes:** Estadísticas generales con gráficas de barras y filtros por fecha, estado y categoría.
- **Administración:** Solo admin — registrar usuarios y asignar roles y contraseñas.

Las relaciones «**include**» indican que un caso de uso siempre ejecuta a otro (ej: crear consulta siempre busca en el KMS). Las relaciones «**extend**» indican comportamiento opcional (ej: los reportes pueden filtrarse por fecha, estado o categoría).

4.4 Modelado de Procesos — Diagramas de Actividad

4.4.1. Flujo de atención de consulta

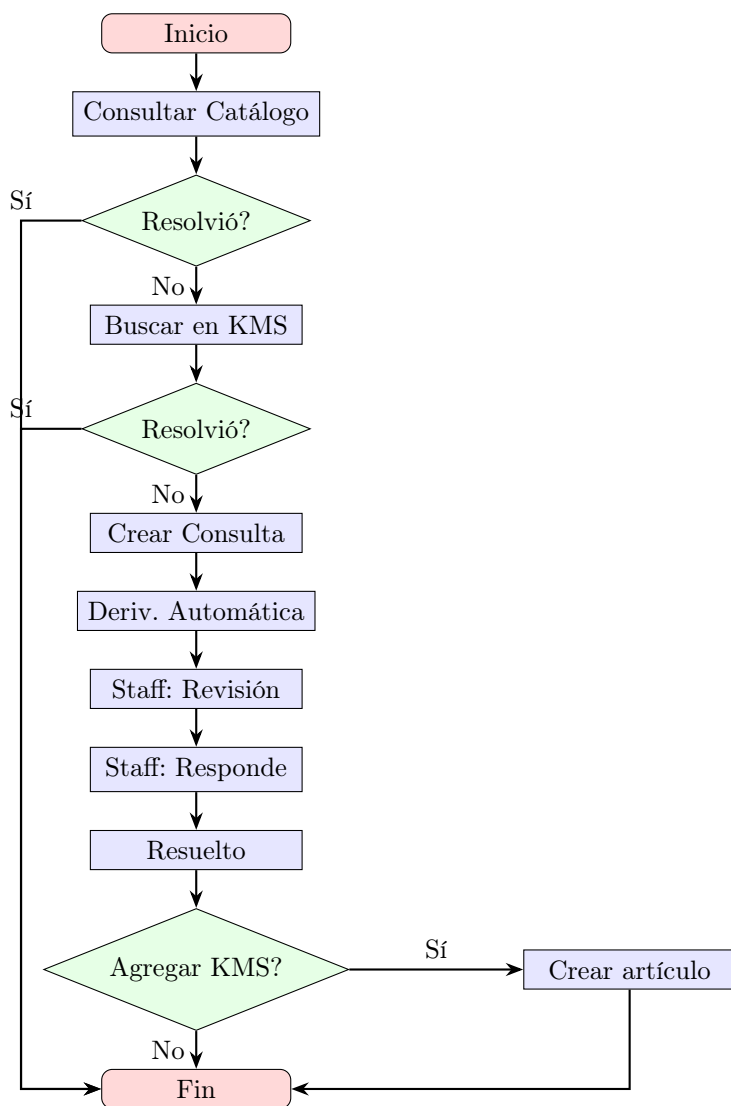


Figura 4: Diagrama de actividad — Flujo completo de atención de consulta

4.4.2. Gestión administrativa

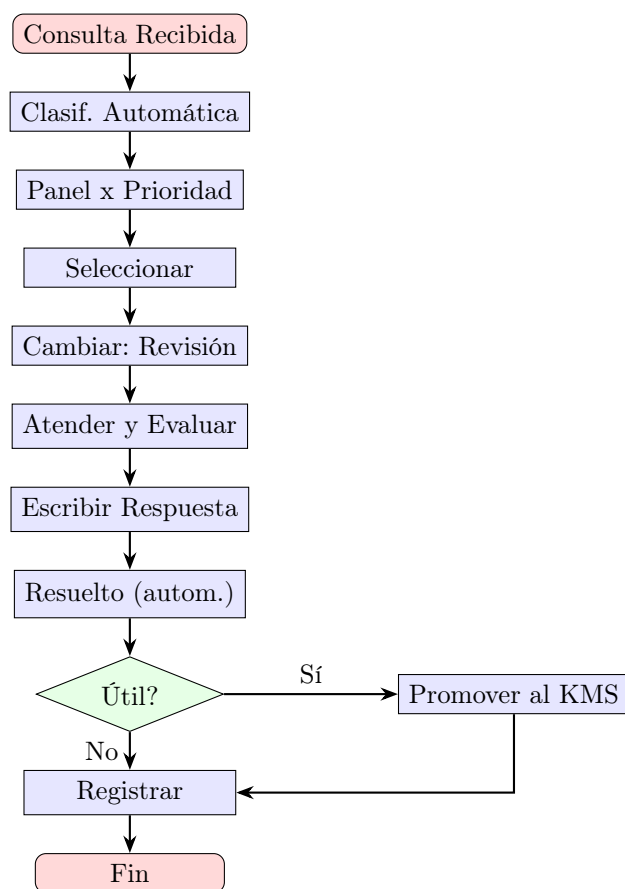


Figura 5: Diagrama de actividad — Gestión administrativa de consultas

4.5 Mockups de pantallas

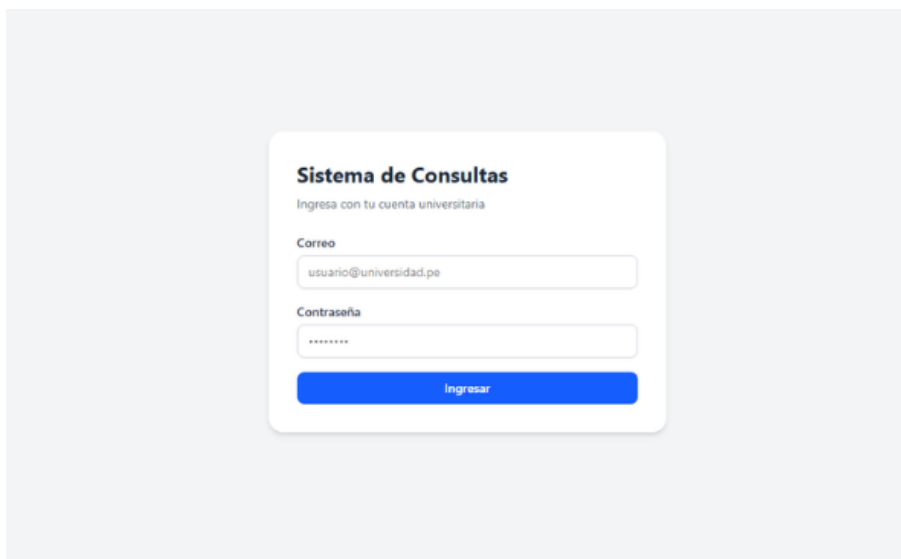


Figura 6: Pantalla de inicio de sesión

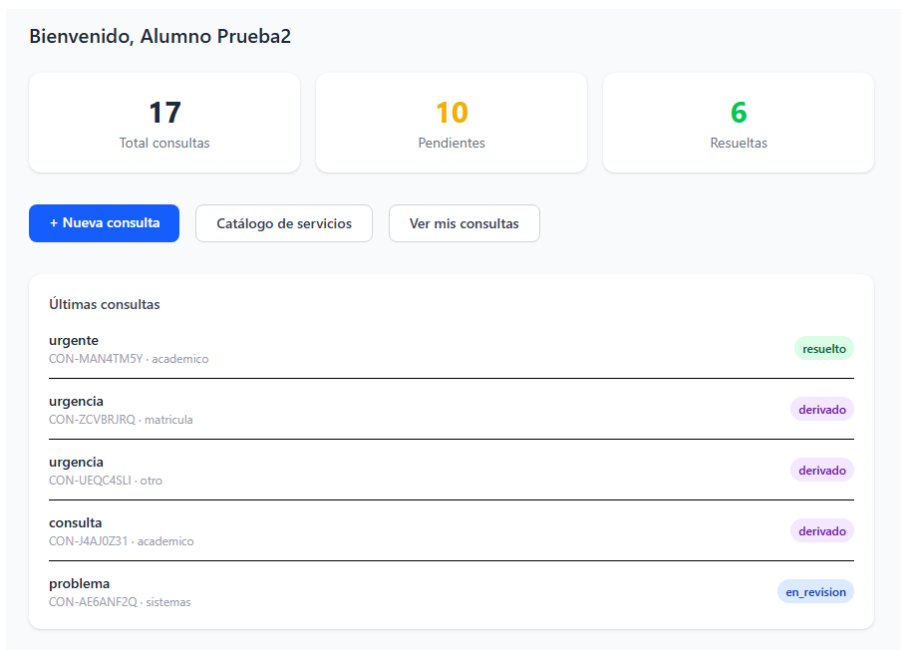


Figura 7: Dashboard del alumno con resumen de consultas

Nueva Consulta

¿Buscas información rápida? Consulta primero el [Catálogo de Servicios](#). Si encuentras lo que necesitas, no hace falta enviar una consulta.

Registrar consulta

Categoría

- academico
- financiero
- infraestructura
- sistemas
- matricula
- tramites
- otro

Las sugerencias se actualizan mientras escribes

Enviar consulta

Figura 8: Formulario de nueva consulta con 7 categorías, banner y enlace al catálogo

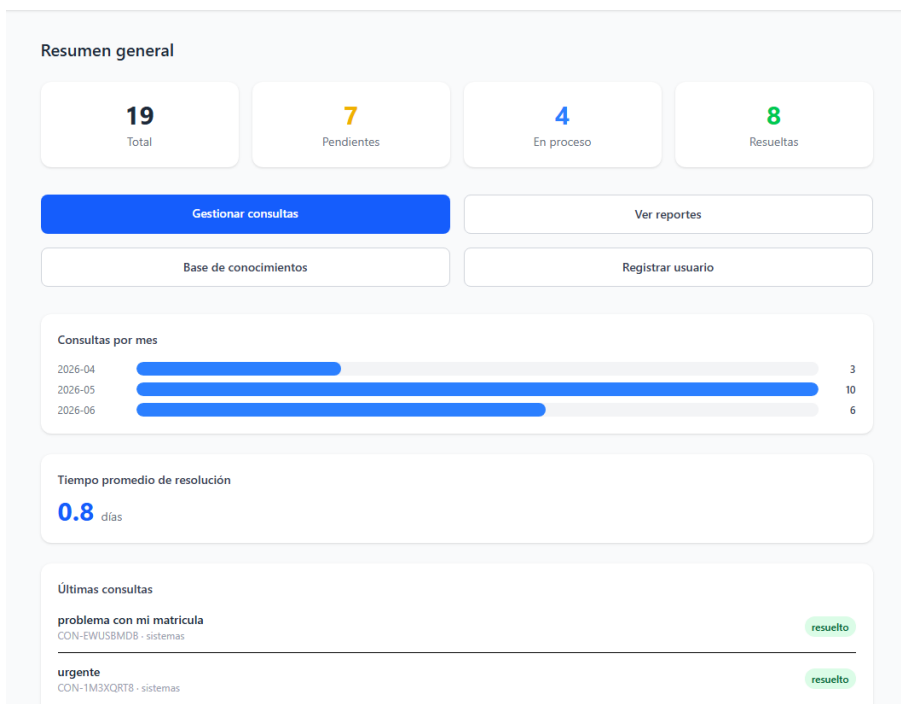


Figura 9: Panel administrativo con KPIs, consultas por mes y tiempo promedio de resolución

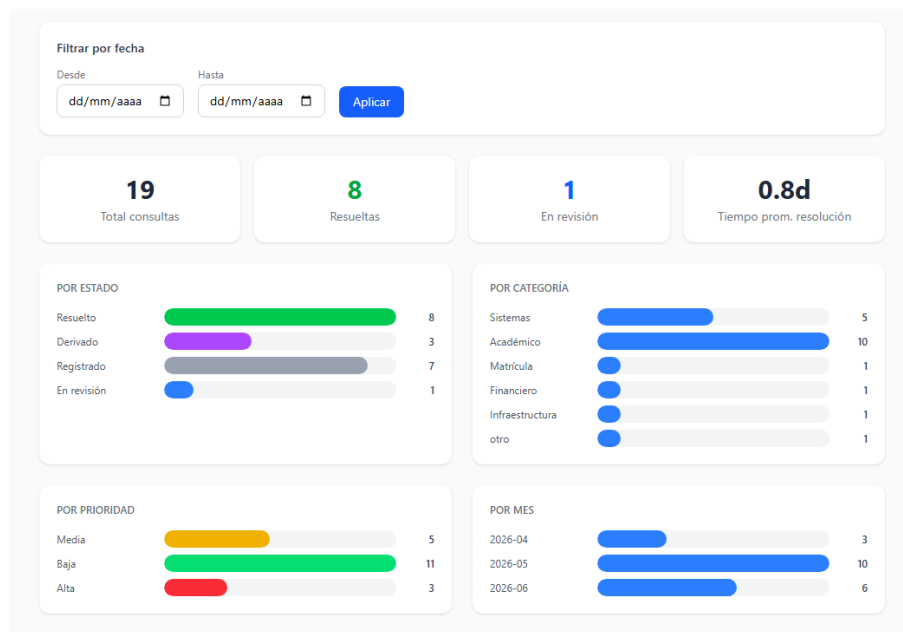


Figura 10: Reportes con gráficas de barras, filtros por fecha/estado/categoría y KPIs

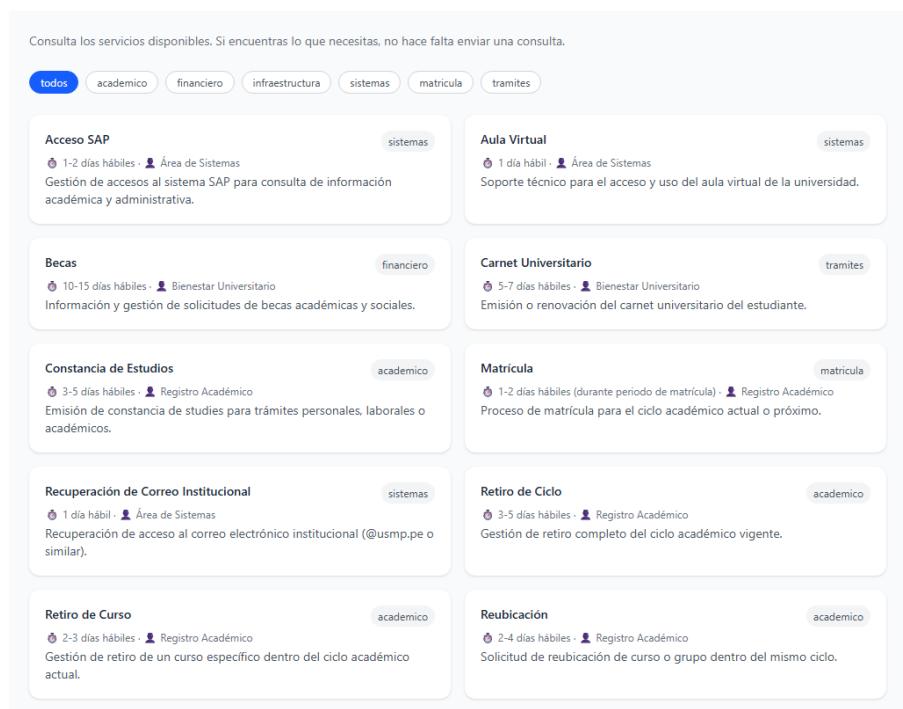


Figura 11: Catálogo de servicios académicos con información detallada de cada trámite

Consulta los servicios disponibles. Si encuentras lo que necesitas, no hace falta enviar una consulta.

todos
academico
financiero
infraestructura
sistemas
matricula
tramites

Acceso SAP sistemas 1-2 días hábiles · Área de Sistemas Gestión de accesos al sistema SAP para consulta de información académica y administrativa.	Aula Virtual sistemas 1 día hábil · Área de Sistemas Soporte técnico para el acceso y uso del aula virtual de la universidad.
Becas financiero 10-15 días hábiles · Bienestar Universitario Información y gestión de solicitudes de becas académicas y sociales.	Carnet Universitario tramites 5-7 días hábiles · Bienestar Universitario Emisión o renovación del carnet universitario del estudiante.
Constancia de Estudios academico 3-5 días hábiles · Registro Académico Emisión de constancia de estudios para trámites personales, laborales o académicos.	Matrícula matricula 1-2 días hábiles (durante periodo de matrícula) · Registro Académico Proceso de matrícula para el ciclo académico actual o próximo.
Recuperación de Correo Institucional sistemas 1 día hábil · Área de Sistemas Recuperación de acceso al correo electrónico institucional (@usmp.pe o similar).	Retiro de Ciclo academico 3-5 días hábiles · Registro Académico Gestión de retiro completo del ciclo académico vigente.
Retiro de Curso academico 2-3 días hábiles · Registro Académico Gestión de retiro de un curso específico dentro del ciclo académico actual.	Reubicación academico 2-4 días hábiles · Registro Académico Solicitud de reubicación de curso o grupo dentro del mismo ciclo.

Figura 12: Catálogo de servicios académicos con información detallada de cada trámite

5. Etapa 3: Desarrollo

5.1 Lenguaje y herramientas usadas

Componente	Tecnología	Uso
Frontend	React 19 + Vite	Interfaz de usuario SPA
Estilos	Tailwind CSS v4	Diseño responsive
Enrutamiento	React Router v7	Navegación por roles
HTTP Client	Axios	Llamadas al backend
Backend	FastAPI (Python)	API REST
Autenticación	Supabase Auth + JWT	Login y control de acceso
Base de datos	Supabase (PostgreSQL)	Almacenamiento de datos
Despliegue Frontend	Vercel	Hosting del frontend
Despliegue Backend	Render	Hosting del backend
Control de versiones	Git + GitHub	Repositorio del código
Contenedores	Docker	Entorno de desarrollo local

5.2 Arquitectura del sistema

El sistema sigue una arquitectura de tres capas desacopladas:

- **Frontend (Vercel):** Aplicación React que consume la API REST mediante Axios. Gestiona el estado de autenticación con AuthContext y protege las rutas según el rol del usuario.
- **Backend (Render):** API REST construida con FastAPI. Organizada en tres capas internas: routers (controladores HTTP), services (lógica de negocio) y data-base (cliente Supabase). Implementa RBAC mediante dependencias de FastAPI.
- **Base de datos (Supabase):** PostgreSQL gestionado en la nube. Supabase Auth maneja la autenticación y emisión de tokens JWT. El backend usa la service role key para acceso completo.

5.3 Capturas del sistema en ejecución

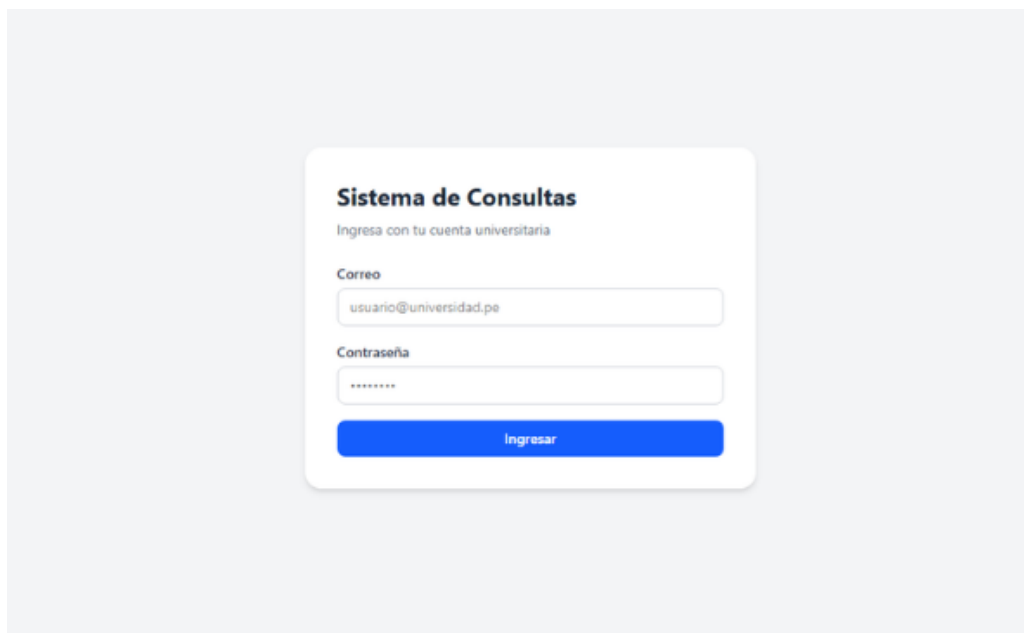


Figura 13: Sistema en ejecución — Pantalla de login

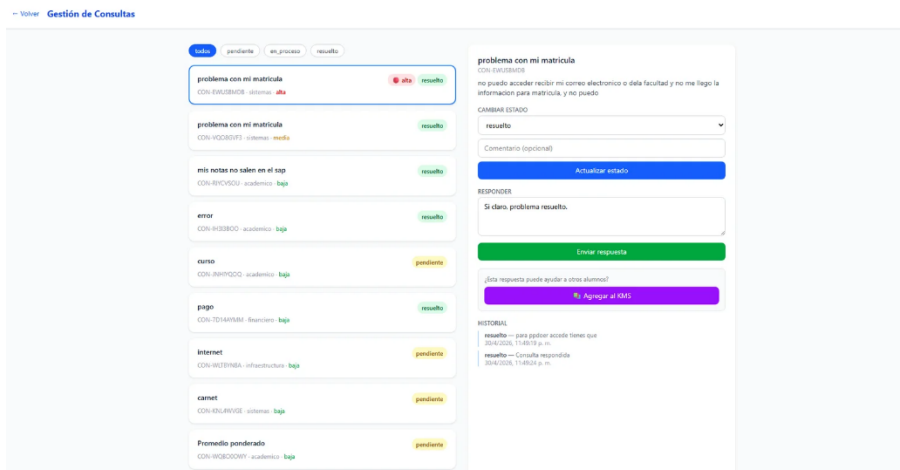


Figura 14: Sistema en ejecución — Gestión de consultas con prioridad

6. Etapa 4: Pruebas realizadas

6.1 Escenarios de prueba

N°	Escenario	Acción	Resultado esperado
1	Login con credenciales válidas	Ingresar email y contraseña correctos	Redirige según rol
2	Login con credenciales inválidas	Ingresar contraseña incorrecta	Muestra mensaje de error
3	Crear consulta con palabra urgente	Escribir “urgente” en descripción	Prioridad asignada: alta
4	KMS sugiere respuesta	Escribir sobre notas en descripción	Aparece artículo relacionado
5	Cambiar estado de consulta	Staff cambia a en_proceso	Estado actualizado e historial registrado
6	Responder consulta	Staff envía respuesta	Estado cambia a resuelto automáticamente
7	Promover respuesta al KMS	Click en “Agregar al KMS”	Artículo creado en base de conocimientos
8	Generar reporte por fechas	Filtrar por rango de fechas	Estadísticas filtradas correctamente
9	Registrar alumno	Admin crea usuario con código 8 dígitos	Contraseña = código de alumno
10	Acceso por rol	Alumno intenta acceder a /admin	Redirige a login
11	Consultar catálogo de servicios	Alumno navega a /alumno/catalogo	Lista de servicios con filtros
12	Derivación automática	Crear consulta categoría “sistemas”	Área asignada: “Área de Sistemas”
13	Categoría “otro”	Crear consulta categoría “otro”	Área asignada: “Secretaría General”
14	Dashboard con métricas	Admin ingresa al dashboard	Muestra consultas por mes y tiempo promedio de resolución
15	Reportes con gráficas	Admin ingresa a reportes	Muestra barras de colores por estado, categoría, prioridad y mes

6.2 Resultados

Todos los escenarios de prueba fueron ejecutados satisfactoriamente. El sistema respondió correctamente en cada caso, validando el flujo completo desde el registro de consultas hasta su resolución y promoción al KMS.

7. Etapa 5: Documentación

7.1 Manual técnico (instalación)

Requisitos previos:

- Python 3.11 o superior
- Node.js 22 o superior
- Cuenta en Supabase con el schema ejecutado

Backend:

```
cd backend
pip install -r requirements.txt
# Crear archivo .env con:
# SUPABASE_URL=...
# SUPABASE_KEY=...
# SUPABASE_JWT_SECRET=...
uvicorn app.main:app --reload --port 8000
```

Frontend:

```
cd frontend
npm install
# Crear archivo .env con:
# VITE_API_URL=http://localhost:8000
npm run dev
```

Con Docker:

```
docker-compose up --build
```

7.2 Manual de usuario

Para el alumno:

1. Ingresar a la URL del sistema con su email y código de alumno como contraseña.
2. Desde el dashboard, hacer click en “Catálogo de Servicios” para consultar información de trámites frecuentes.

3. Si la información del catálogo o del KMS no resuelve la duda, hacer click en “+ Nueva consulta”.
4. Seleccionar la categoría, escribir el título y descripción. El sistema sugerirá artículos del KMS automáticamente.
5. Si ninguna sugerencia resuelve la duda, hacer click en “Enviar consulta”.
6. En “Mis consultas” puede ver el estado actual, el área derivada y la respuesta del staff.
7. Al hacer clic en una consulta puede ver el historial completo de cambios de estado.

Para el staff (secretaria/admin):

1. Ingresar con las credenciales asignadas por el administrador.
2. En “Gestionar consultas” ver la lista ordenada por prioridad (alta primero) y filtrable por estado.
3. Las consultas ya llegan derivadas al área responsable automáticamente según la categoría seleccionada por el alumno.
4. Seleccionar una consulta para ver el detalle, cambiar su estado o responderla.
5. Al responder, el estado cambia automáticamente a resuelto.
6. Si la respuesta puede ayudar a otros alumnos, hacer click en “Agregar al KMS”.
7. En “Reportes” generar estadísticas visuales con gráficas de barras por estado, categoría, prioridad y mes, filtrando por fechas si es necesario.

8. Etapa 6: Propuesta de mejoras / mantenimiento

- **Búsqueda semántica en KMS:** Reemplazar la búsqueda por palabras exactas con búsqueda semántica usando modelos de lenguaje (NLP), permitiendo encontrar artículos aunque el alumno use sinónimos o frases distintas.
- **Prioridad con Machine Learning:** Mejorar la inferencia de prioridad usando modelos de clasificación de texto entrenados con el historial de consultas del sistema.
- **Notificaciones automáticas:** Enviar notificaciones por email o WhatsApp al alumno cuando su consulta cambie de estado o reciba respuesta.
- **Asignación automática de staff:** Asignar automáticamente cada consulta al miembro del staff responsable dentro del área derivada.
- **Encuesta de satisfacción:** Agregar una encuesta al alumno después de resolver su consulta para medir la calidad de la atención.
- **Plantillas de respuesta:** Permitir que el staff cree y use plantillas de respuestas predefinidas para consultas frecuentes.
- **Aplicación móvil:** Desarrollar una versión móvil nativa para mejorar la accesibilidad desde dispositivos Android e iOS.

9. Conclusiones y aprendizajes

El desarrollo del Sistema de Gestión Administrativa para Consultas permitió aplicar de forma integral los conocimientos adquiridos en el curso, abarcando desde el análisis de requerimientos hasta el despliegue en producción.

- Se logró centralizar la gestión de consultas estudiantiles en una plataforma digital accesible desde cualquier dispositivo con conexión a internet.
- La implementación de prioridad automática por análisis de palabras clave demostró ser efectiva para ordenar la atención del personal administrativo, priorizando los casos más urgentes.
- La Base de Conocimientos (KMS) redujo la necesidad de consultas repetitivas al sugerir respuestas automáticas en tiempo real mientras el alumno escribe.
- El Catálogo de Servicios Académicos proporciona información detallada de trámites frecuentes, permitiendo que los alumnos resuelvan dudas sin necesidad de crear una consulta.
- La derivación automática por categoría eliminó la clasificación manual, asignando cada consulta al área responsable de forma transparente para el alumno.
- El sistema de roles y autenticación JWT garantizó que cada usuario acceda únicamente a las funciones que le corresponden.
- El historial de estados proporcionó trazabilidad completa de cada consulta, facilitando la auditoría y el seguimiento.
- Las métricas temporales (consultas por mes y tiempo promedio de resolución) y las gráficas visuales en reportes brindan al staff herramientas para la toma de decisiones.
- El despliegue en la nube (Vercel + Render + Supabase) permitió que el sistema sea accesible sin necesidad de infraestructura local.
- Se aprendió a trabajar con una arquitectura desacoplada de tres capas, separando responsabilidades entre frontend, backend y base de datos.

10. Anexos

10.1 Código fuente

El código fuente completo del proyecto está disponible en el repositorio de GitHub:
<https://github.com/Vakipandi/ProyectoFinalSist>

El sistema está desplegado y accesible en:

Frontend: <https://proyecto-final-sist.vercel.app>

Backend (API): <https://proyectofinal-backend-mfhl.onrender.com/docs>

10.2 Estructura de base de datos

Tabla	Campo clave	Descripción
users	id (UUID)	Usuarios del sistema con rol asignado
service_catalog	id (UUID)	Catálogo de servicios académicos con nombre, descripción, requisitos y procedimiento
consultations	code (CON-XXXXXXXX)	Consultas con prioridad inferida, área derivada y estado
status_history	consultation_id	Auditoría de cambios de estado
kms_articles	keywords[]	Artículos de la base de conocimientos

10.3 Bibliografía

- FastAPI Documentation. *FastAPI - Modern, fast web framework for building APIs*. Recuperado de <https://fastapi.tiangolo.com>
- Supabase Documentation. *Supabase - The Open Source Firebase Alternative*. Recuperado de <https://supabase.com/docs>
- React Documentation. *React - The library for web and native user interfaces*. Recuperado de <https://react.dev>
- Vite Documentation. *Vite - Next Generation Frontend Tooling*. Recuperado de <https://vitejs.dev>
- Tailwind CSS Documentation. *Tailwind CSS - A utility-first CSS framework*. Recuperado de <https://tailwindcss.com>
- Vercel Documentation. *Vercel - Develop. Preview. Ship*. Recuperado de <https://vercel.com/docs>